

- ❑ Topic of the session: Advanced Java Programming
- ❑ UNIT 5: JavaBeans
- ❑ **University of Kigali (UoK)**
- ❑ Trainer name: Dr. NTEZIRIZA NKERABAHIZI Josbert

JavaBeans



5. 1. Introduction

5

- **JavaBeans are special Java classes (components) that follow certain rules so they can be easily reused, managed, and manipulated inside programs or tools like NetBeans.**
- **A Java Bean is a reusable software component that can be visually manipulated in builder tools.**

5. 1. Introduction

5

A **JavaBean** is a reusable Java class that:

- Has **private properties (variables)**.
- Uses **getter and setter methods**.
- Has a **no-argument constructor**.
- Can support features like **events and persistence**.

```
public class Student {  
    private String name; // property  
  
    public Student() { } // no-argument constructor  
  
    public String getName() { // getter  
        return name;  
    }  
  
    public void setName(String name) { // setter  
        this.name = name;  
    }  
}
```

5. 1. Introduction

5

■ Encapsulation

- Data is hidden (private variables) and access through methods (get/set).

■ Reusability

- Can be used in many programs.

■ Properties

- Variables like name, age.

■ Events

- Can notify changes (like your User example using **PropertyChangeSupport**).

■ Persistence: Can be saved and loaded.

5. 1. Introduction

5

Rules of JavaBeans

To be a JavaBean, a class must:

- Have a **public no-argument constructor**.
- Use **private variables**.
- Provide **getter and setter methods**.
- (Optional) implement **Serializable**.

5.2. Why implement Serializable?

In JavaBeans, **Serializable** is used to allow an object to be:

- **Saved to a file or transferred over a network and later returned.**
- When a JavaBean implements **Serializable**, it means:
- You can **convert the object into bytes** (serialization).
- Store it in:
 - File
 - Database
 - Network (send to another system).
- Then **rebuild it later** (deserialization).

5.2. Why it is Important in JavaBeans

5

- **Persistence (Saving Data)**

- JavaBeans can be saved and loaded later.
- Example: saving user data.

- **Used in GUI Builders (NetBeans)**

- Tools store component states as serialized object.

- **Web Applications**

- Beans can be stored in sessions and transferred between servers.

- **Easy Data Transfer**

- Send objects across networks (e.g., client ↔ server).

Example:

5

```
import java.io.Serializable;

public class Student implements Serializable {
    private String name;

    public Student() {}

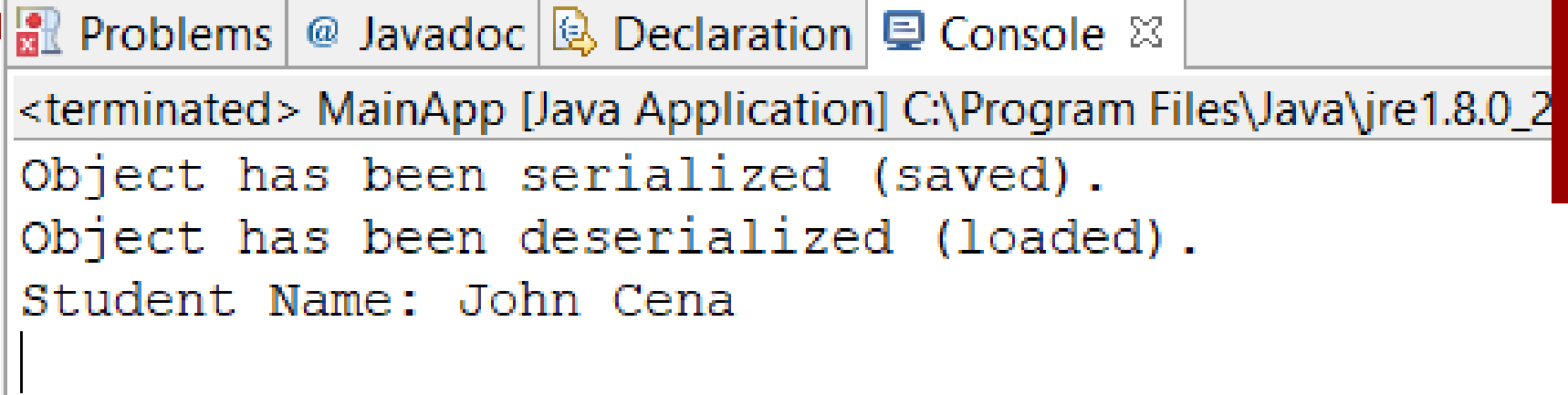
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

```
1 package Dr_Josbert_JavaBeans;
2
3 import java.io.*; // Import classes for input/output operations
4
5 //JavaBean class that can be saved (serialized)
6 class Student implements Serializable {
7
8     // Unique ID for serialization (avoids version mismatch errors)
9     private static final long serialVersionUID = 1L;
10
11     // Private variable (property)
12     private String name;
13
14     // No-argument constructor (required for JavaBeans)
15     public Student() {}
16
17     // Getter method to access the name
18     public String getName() {
19         return name;
20     }
21
22     // Setter method to modify the name
23     public void setName(String name) {
24         this.name = name;
25     }
26 }
```

```
1 package Dr_Josbert_JavaBeans;
2
3 import java.io.FileInputStream;
4 import java.io.FileOutputStream;
5 import java.io.IOException;
6 import java.io.ObjectInputStream;
7 import java.io.ObjectOutputStream;
8
9 //Main class where execution starts
10 public class MainApp {
11
12 public static void main(String[] args) {
13
14     // -----
15     // STEP 1: Create Student object
16     // -----
17     Student student = new Student();
18     student.setName("John Cena"); // Set student name
19
20     // -----
21     // STEP 2: SERIALIZATION (Saving object to file)
22     // -----
23     try {
24         // Create file output stream (file will be created if it doesn't exist)
25         FileOutputStream fileOut = new FileOutputStream("student.ser");
26
27         // Create object output stream to write object into file
28         ObjectOutputStream out = new ObjectOutputStream(fileOut);
29
30         // Write (serialize) the student object into file
31         out.writeObject(student);
32
33         // Close streams to avoid memory leaks
34         out.close();
35         fileOut.close();
36 
```

```
37 // Message to confirm saving
38 System.out.println("Object has been serialized (saved).");
39
40 } catch (IOException e) {
41     // Handle errors during writing
42     e.printStackTrace();
43 }
44
45 // -----
46 // STEP 3: DESERIALIZATION (Reading object from file)
47 // -----
48 try {
49     // Open file input stream to read the file
50     FileInputStream fileIn = new FileInputStream("student.ser");
51
52     // Create object input stream to read object
53     ObjectInputStream in = new ObjectInputStream(fileIn);
54
55     // Read (deserialize) object from file and convert back to Student
56     Student loadedStudent = (Student) in.readObject();
57
58     // Close streams
59     in.close();
60     fileIn.close();
61
62     // Display confirmation and data
63     System.out.println("Object has been deserialized (loaded).");
64     System.out.println("Student Name: " + loadedStudent.getName());
65
66 } catch (IOException | ClassNotFoundException e) {
67     // Handle errors during reading
68     e.printStackTrace();
69 }
70 }
```



The screenshot shows an IDE window with a tab bar at the top containing 'Problems', 'Javadoc', 'Declaration', and 'Console'. The 'Console' tab is active, displaying the following text: '<terminated> MainApp [Java Application] C:\Program Files\Java\jre1.8.0_2', 'Object has been serialized (saved).', 'Object has been deserialized (loaded).', and 'Student Name: John Cena'. A vertical cursor is positioned at the end of the last line.

```
<terminated> MainApp [Java Application] C:\Program Files\Java\jre1.8.0_2
Object has been serialized (saved).
Object has been deserialized (loaded).
Student Name: John Cena
|
```

5. 3. Where JavaBeans Are Used?

5

- JavaBeans are used in many areas of Java programming because they are **simple, reusable, and easy to manage.**

1. GUI Applications (Desktop Apps)

- Used in tools like **NetBeans GUI Builder.**
- You can drag and drop components (buttons, text fields).
- Example: A form with:
 - Name field
 - Submit button
- Each component can be a JavaBean.

5. 3. Where JavaBeans Are Used?

2. Web Applications (JSP & Servlets)

- JavaBeans store and transfer data between pages.

➤ Example: Login system:

- Username and password stored in a JavaBean.
- Passed between JSP page.

5. 3. Where JavaBeans Are Used?

3. Frameworks (Enterprise Applications)

- Used in popular frameworks like:

- Spring Framework
- Hibernate

➤ Example:

- A JavaBean stores data from a database.

5. 3. Where JavaBeans Are Used?

4. Data Storage (Persistence)

- JavaBeans can be saved to files using the Serializable interface.
- Example:
 - Save student information.
 - Load it later from a file.

5. Networking (Distributed Systems)

- Used to send objects across systems.
- Example: Client sends a JavaBean to server.

5. 3. Where JavaBeans Are Used?

■ 6. Reusable Components

- JavaBeans are designed to be reused in many programs.
- Example:
 - A Student class used in:
 - School system
 - Exam system
 - Registration system

5. 3. Where JavaBeans Are Used?

5

7. Event Handling

- JavaBeans can respond to changes using listeners.
- Example: When a property changes → activate an event.

Summary : JavaBeans are used in:

- GUI applications
- Web applications
- Frameworks
- Data storage
- Networking
- Reusable components

5.4. Advantages of JavaBeans

5

1. Reusability

- JavaBeans can be used in different programs without rewriting code.
- Example: A `Student` bean used in many systems.

2. Encapsulation

- Data is hidden using **private variables**.
- Access is controlled using **getters and setters**.
- Improves **security and data control**.

5.4. Advantages of JavaBeans

5

3. Easy Maintenance

- Code is well-structured and easy to update.
- Changes in one part do not affect others.

4. Supports Visual Tools

- Works with GUI builders like NetBeans.
- Allows **drag-and-drop design** (is a way of creating interfaces by clicking and placing components visually instead of writing code.)
- Faster application development.

5.4. Advantages of JavaBeans

5

5. Persistence (Saving Data)

- JavaBeans can be saved and loaded using Serializable.
- Useful for storing data in files or databases.

6. Event Handling Support

- JavaBeans can respond to changes using listeners.
- Example: When a value changes → event is activated.

5.4. Advantages of JavaBeans

5

7. Platform Independence

- JavaBeans run on any system that supports Java
- “Write once, run anywhere”.

8. Easy Integration

- Works well with frameworks like Spring Framework and Hibernate.

Summary

5

Advantages of JavaBeans:

- Reusable
- Encapsulated
- Easy to maintain
- Supports GUI tools
- Supports persistence
- Event handling
- Platform independent

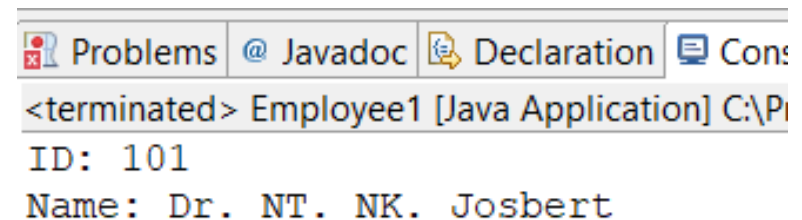
JavaBean Simple Implementation:

5

```
1 package Dr_Josbert_JavaBeans;
2
3 import java.io.Serializable;
4
5 public class Employee implements Serializable
6 {
7     private int id;
8     private String name;
9
10    public Employee()
11    {
12    }
13
14    public void setId(int id)
15    {
16        this.id = id;
17    }
18
19    public int getId()
20    {
21        return id;
22    }
23
24    public void setName(String name)
25    {
26        this.name = name;
27    }
28
29    public String getName()
30    {
31        return name;
32    }
33 }
```

```
1 package Dr_Josbert_JavaBeans;
2
3 public class Employee1 {
4     public static void main(String args[])
5     {
6         Employee s = new Employee();
7
8         s.setId(101);
9         s.setName("Dr. NT. NK. Josbert");
10
11         System.out.println("ID: " + s.getId());
12         System.out.println("Name: " + s.getName());
13     }
14 }
```

Output:



```
<terminated> Employee1 [Java Application] C:\P
ID: 101
Name: Dr. NT. NK. Josbert
```

5.5. What is BeanInfo Interface?

5

- **BeanInfo** is a **Java interface** used with **JavaBeans** to **control how the bean is presented to development tools** (like NetBeans, Eclipse, or GUI builders).
- Normally, JavaBeans use **self-analysis** to automatically find:
 - Properties (variables with getters/setters)
 - Methods
 - Events
- But sometimes, you want to **customize what the tools see**.
- This is exactly what BeanInfo does: it **tells the tool which properties, methods, and events to display/show**.

Key Points

- **Purpose:** Customize the visibility of bean features in development tools.
- **Use Case:** Hide some properties, rename them, or change their order.
- **Part of:** java.beans package.

Example Conceptually

- Imagine a Student JavaBean with properties:

- Name

- Age

By default, tools will show both.

- Using BeanInfo, you can **show only Name, hiding Age.**

Methods in BeanInfo

- `getPropertyDescriptors()` → Control which properties are visible.
- `getMethodDescriptors()` → Control which methods are visible.
- `getEventSetDescriptors()` → Control which events are visible.
- `getBeanDescriptor()` → Provides information about the bean as a whole.
- `getIcon(int iconKind)` → Optional icons for the bean.

Real-Life Analogy

- Think of **BeanInfo** as a menu for a restaurant:
 - The kitchen (JavaBean) has many dishes (properties/methods/events).
 - The menu (BeanInfo) decides which dishes are visible to customers (tools).
- BeanInfo is a Java interface that allows developers to customize how a JavaBean is displayed and interacted with in development tools.

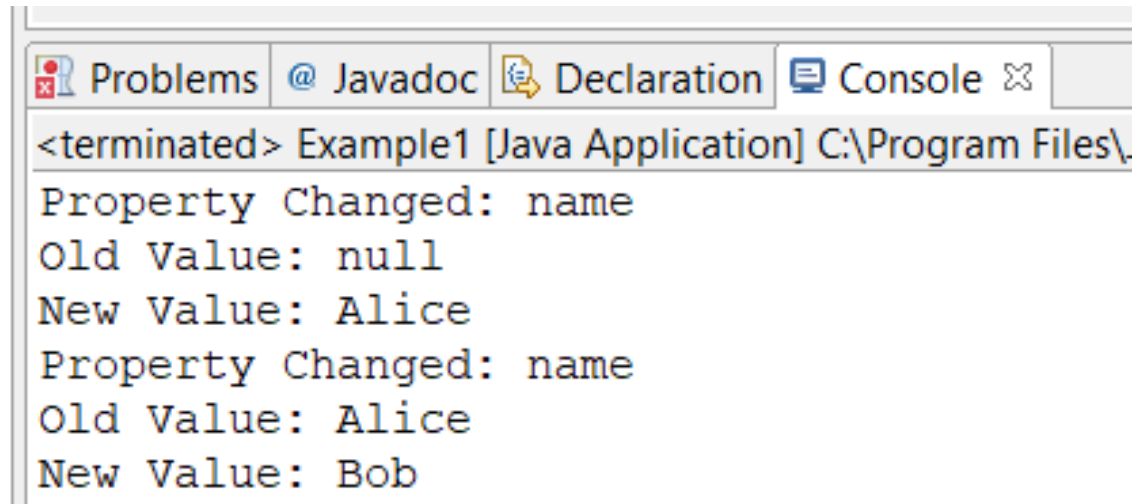
Example 1: JavaBean with Property Change Listener

5

```
2
3 import java.beans.*;
4
5 //-----
6 //JavaBean class
7 //-----
8 class User {
9     private String name; // private property (encapsulation)
10
11     // PropertyChangeSupport helps notify listeners about changes
12     private final PropertyChangeSupport support = new PropertyChangeSupport(this);
13
14     // No-argument constructor (required for JavaBeans)
15     public User() {}
16
17     // Getter for 'name' property
18     public String getName() {
19         return name;
20     }
21
22     // Setter for 'name' property with event notification
23     public void setName(String name) {
24         String oldName = this.name; // store old value
25         this.name = name;           // set new value
26         support.firePropertyChange("name", oldName, name); // notify listeners
27     }
28
29     // Method to add listener
30     public void addPropertyChangeListener(PropertyChangeListener listener) {
31         support.addPropertyChangeListener(listener);
32     }
33
34     // Method to remove listener
35     public void removePropertyChangeListener(PropertyChangeListener listener) {
36         support.removePropertyChangeListener(listener);
37     }
38 }
```

```
1 package Dr_Josbert_JavaBeans;
2
3 public class Example1 {
4     public static void main(String[] args) {
5         User user = new User(); // create JavaBean object
6
7         // Add listener to track changes in 'name' property
8         user.addPropertyChangeListener(event -> {
9             System.out.println("Property Changed: " + event.getPropertyName());
10            System.out.println("Old Value: " + event.getOldValue());
11            System.out.println("New Value: " + event.getNewValue());
12        });
13
14        // Changing property (triggers listener)
15        user.setName("Alice");
16        user.setName("Bob");
17    }
18 }
```

Output:



The screenshot shows an IDE window with tabs for Problems, Javadoc, Declaration, and Console. The Console tab is active, displaying the output of the Java application. The output consists of two sets of three lines each, corresponding to the two setName calls in the code. The first set shows the initial state where the name is null, and the second set shows the state after the name has been changed from Alice to Bob.

```
<terminated> Example1 [Java Application] C:\Program Files\
Property Changed: name
Old Value: null
New Value: Alice
Property Changed: name
Old Value: Alice
New Value: Bob
```


Role of `addPropertyChangeListener` and `removePropertyChangeListener`

- JavaBeans often support **events**, which allow objects to **notify other objects when something changes**.
- The **`PropertyChangeListener`** is used to **listen for changes in properties** of a `JavaBean`.

1. `addPropertyChangeListener`

```
public void addPropertyChangeListener(PropertyChangeListener listener) {  
    support.addPropertyChangeListener(listener);  
}
```

Role of `addPropertyChangeListener` and `removePropertyChangeListener`

Role:

- Registers a listener to the bean.
- The listener **will be notified** whenever a property changes.
- **Used when you want to monitor changes in the bean's data.**

Example:

```
user.addPropertyChangeListener(event -> {  
    System.out.println("Property " + event.getPropertyName() + " changed!");  
});
```

This listener **activates whenever** `user.setName()` is called

Role of addPropertyChangeListener and removePropertyChangeListener

2. removePropertyChangeListener

```
public void removePropertyChangeListener(PropertyChangeListener listener) {  
    support.removePropertyChangeListener(listener);  
}
```

- **Role:**
- **Unregisters a listener** from the bean.
- The listener will **no longer be notified** when properties change.
- **Prevents unnecessary notifications** and avoids memory leaks.

```
user.removePropertyChangeListener(myListener);
```

After this, changes to user.name **won't trigger myListener anymore**

Role of `addPropertyChangeListener` and `removePropertyChangeListener`

Method	Role
<code>addPropertyChangeListener</code>	Subscribe to the radio → you start receiving notifications (music/news)
<code>removePropertyChangeListener</code>	Unsubscribe → stop receiving notifications

- `addPropertyChangeListener` → Adds a listener to **watch for property changes**.
- `removePropertyChangeListener` → Removes a listener to **stop watching property changes**.
- Both methods allow **event-driven programming in JavaBeans**.

Example 2: JavaBean with BeanInfo Interface (custom properties)

5

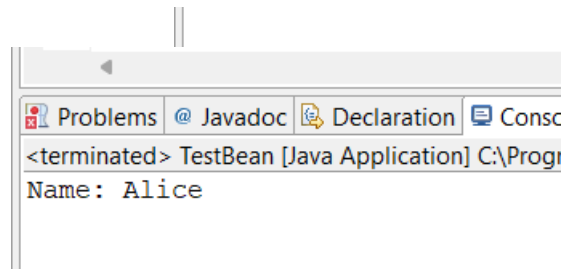
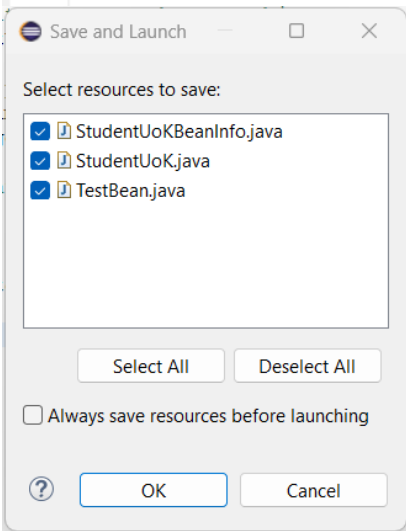
```
1 package Dr_Josbert_JavaBeans;
2
3 import java.io.Serializable;
4
5 //JavaBean class representing a Student
6 public class StudentUoK implements Serializable {
7
8     // Private properties (fields)
9     private String name;
10    private int age; // This property will be hidden in BeanInfo
11
12    // Default constructor
13    public StudentUoK() {}
14
15    // Getter and setter for 'name'
16    public String getName() {
17        return name;
18    }
19
20    public void setName(String name) {
21        this.name = name;
22    }
23
24    // Getter and setter for 'age'
25    public int getAge() {
26        return age;
27    }
28
29    public void setAge(int age) {
30        this.age = age;
31    }
32 }
```

```
1 package Dr_Josbert_JavaBeans;
2
3 import java.beans.*;
4
5 //BeanInfo class to customize property visibility for GUI tools
6 public class StudentUoKBeanInfo extends SimpleBeanInfo {
7
8     // Only show the 'name' property; 'age' will be hidden
9     @Override
10    public PropertyDescriptor[] getPropertyDescriptors() {
11        try {
12            // Create a PropertyDescriptor for 'name' only
13            PropertyDescriptor nameProp = new PropertyDescriptor("name", StudentUoK.class);
14
15            // Return only 'name' property
16            return new PropertyDescriptor[] { nameProp };
17
18        } catch (IntrospectionException e) {
19            e.printStackTrace();
20            return null;
21        }
22    }
23 }
```

```

1 package Dr_Josbert_JavaBeans;
2
3 public class TestBean {
4     public static void main(String[] args) {
5
6         // Create a new StudentUoK object
7         StudentUoK s = new StudentUoK();
8
9         // Set properties
10        s.setName("Alice"); // visible property
11        s.setAge(20);        // hidden property (won't appear in GUI tools)
12
13        // Print visible property
14        System.out.println("Name: " + s.getName());
15
16        // The 'age' property is hidden in BeanInfo but still accessible in code
17        // System.out.println("Age: " + s.getAge()); // optional
18    }
19 }
20

```



5.6. JavaBeans in GUI

```
1 package Dr_Josbert_JavaBeans;
2
3 import java.awt.*;           // Used for GUI components (Canvas, Frame, Color, Graphics)
4 import java.io.Serializable; // Used to make the class a JavaBean (Serializable)
5
6 // JavaBean class that extends Canvas to draw custom shapes
7 public class FirstBeanShapes extends Canvas implements Serializable {
8
9     // Constructor: runs when object is created
10    public FirstBeanShapes() {
11
12        // Set the width and height of the drawing area (canvas)
13        setSize(200, 200);
14
15        // Set background color of the canvas
16        setBackground(Color.white);
17    }
18
19    // paint() method is automatically called to draw graphics
20    @Override
21    public void paint(Graphics g) {
22
23        // =====
24        // RECTANGLE (RED) - Positioned at TOP LEFT
25        // =====
26        g.setColor(Color.RED);           // Set drawing color to red
27        g.fillRect(20, 20, 80, 40);      // x, y, width, height
28    }
```



```
29 // =====
30 // CIRCLE (GREEN) - Positioned at RIGHT SIDE (MIDDLE)
31 // =====
32 g.setColor(Color.GREEN); // Set color to green
33 g.fillOval(110, 70, 60, 60); // x, y, width, height
34
35 // =====
36 // TRIANGLE (BLUE) - Positioned at BOTTOM LEFT
37 // =====
38 g.setColor(Color.BLUE); // Set color to blue
39
40 // x-coordinates of triangle corners
41 int[] xPoints = {40, 10, 70};
42
43 // y-coordinates of triangle corners
44 int[] yPoints = {150, 190, 190};
45
46 // Draw filled triangle using 3 points
47 g.fillPolygon(xPoints, yPoints, 3);
48 }
49
```

```
50 // Main method: program execution starts here
51 public static void main(String[] args) {
52
53     // Create a window (Frame) to display the canvas
54     Frame frame = new Frame("Separated Shapes Bean");
55
56     // Create object of our JavaBean class
57     FirstBeanShapes bean = new FirstBeanShapes();
58
59     // Set size of the window (width, height)
60     frame.setSize(300, 300);
61
62     // Disable default layout manager (manual positioning used)
63     frame.setLayout(null);
64
65     // Set position of canvas inside the frame
66     bean.setLocation(50, 50);
67
68     // Add canvas (bean) to the frame
69     frame.add(bean);
70
71     // Make the window visible on screen
72     frame.setVisible(true);
73
74     // Add window closing functionality
75     frame.addWindowListener(new java.awt.event.WindowAdapter() {
76         public void windowClosing(java.awt.event.WindowEvent e) {
77             System.exit(0); // Exit program when window is closed
78         }
79     });
80 }
```

